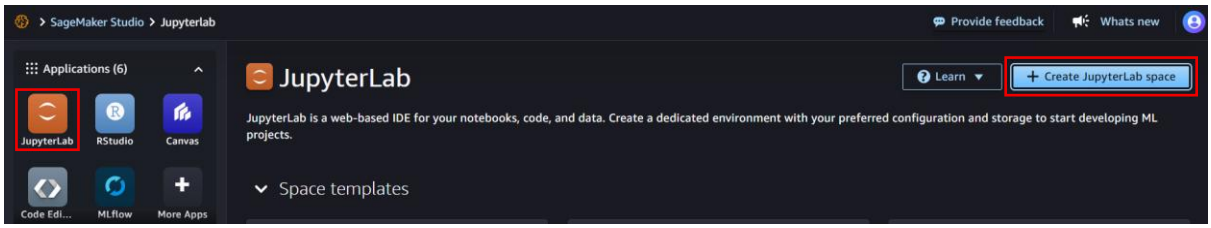
Training the model

To Begin with the Lab

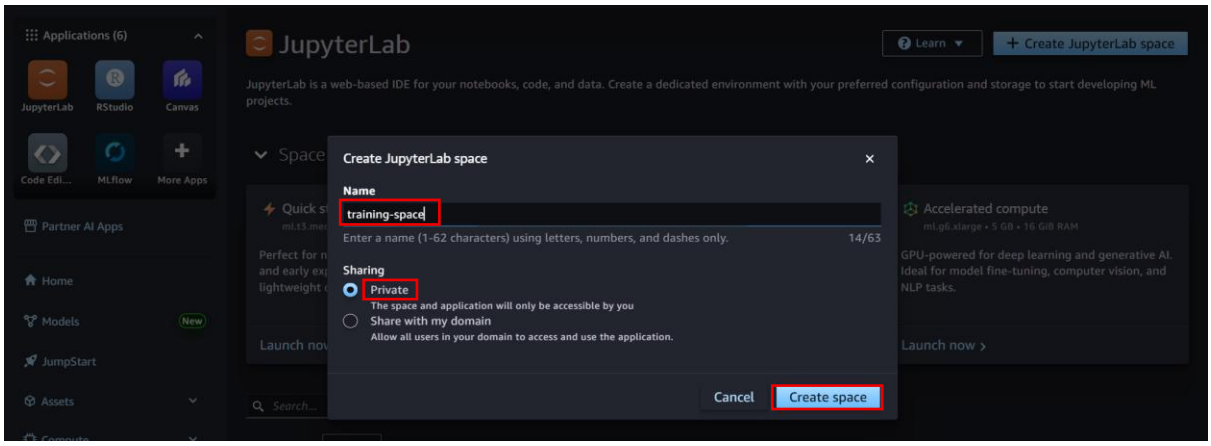
Summary of the Lab

This lab demonstrates how to build an employee attrition prediction model using Amazon SageMaker.A private JupyterLab space named test-space was created in SageMaker Studio, where the employee dataset and training notebook were uploaded. The dataset was cleaned and preprocessed by converting categorical values into numerical format, then stored in SageMaker Feature Store. Using Scikit-learn, the dataset was divided into training and validation sets and uploaded to Amazon S3. An XGBoost model was trained in SageMaker using the processed data. Hyperparameter tuning was configured to test multiple parameter combinations, identify the best-performing training job, and retrieve the optimal hyperparameters for future model use.

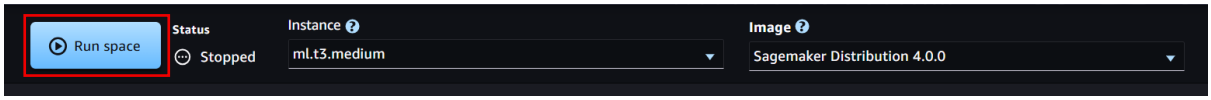
- Open Amazon SageMaker Studio.
- Navigate to JupyterLab.
- Click Create Space.



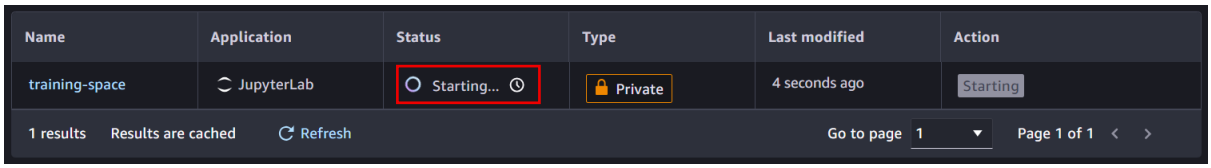
- Enter:
- Name: test-space
- Type: Private
- Click Create
- Start the space and wait until status is Running



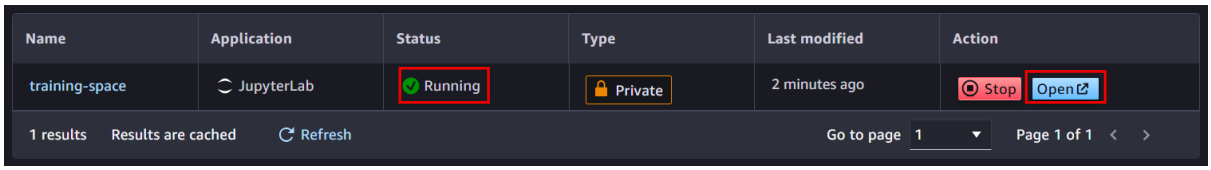
- Click Run Space



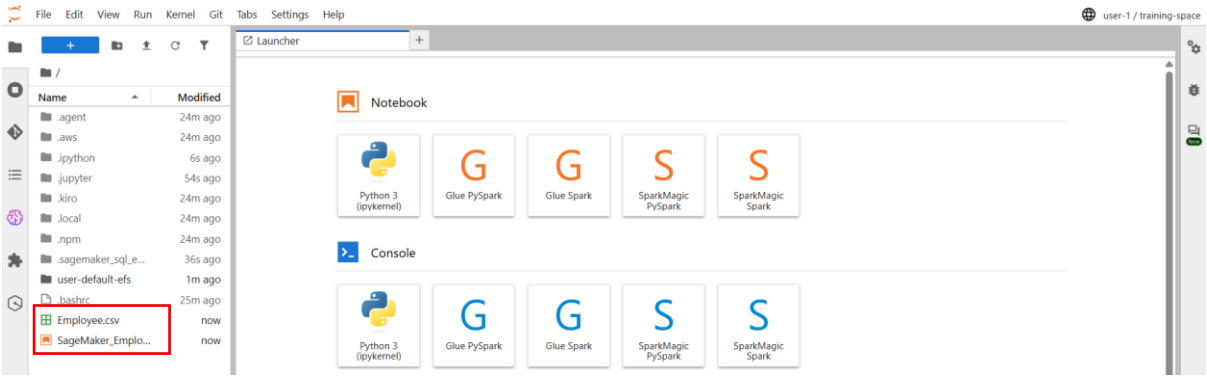
- Click Open JupyterLab once space is ready.



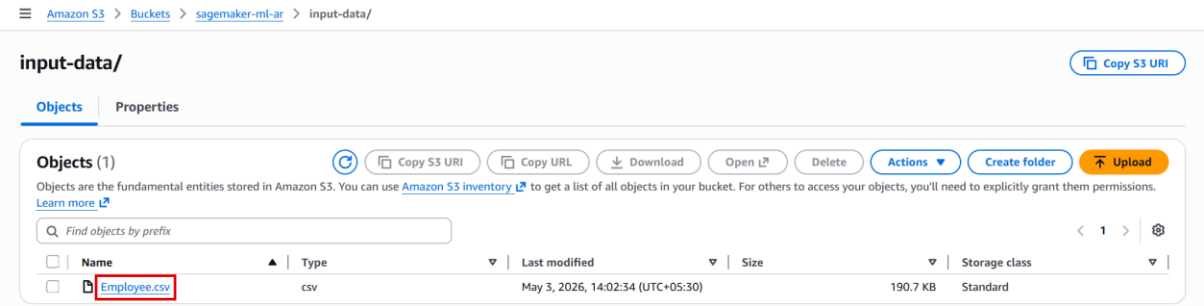
- Click open



- We have employee dataset where each row represents an employee with features like education, joining year, city, payment tier, and age.
- The goal is to predict whether an employee will leave (1) or stay (0), making it a binary classification problem.
- Inside JupyterLab:
- Upload:
- The dataset (CSV file)
- The training notebook



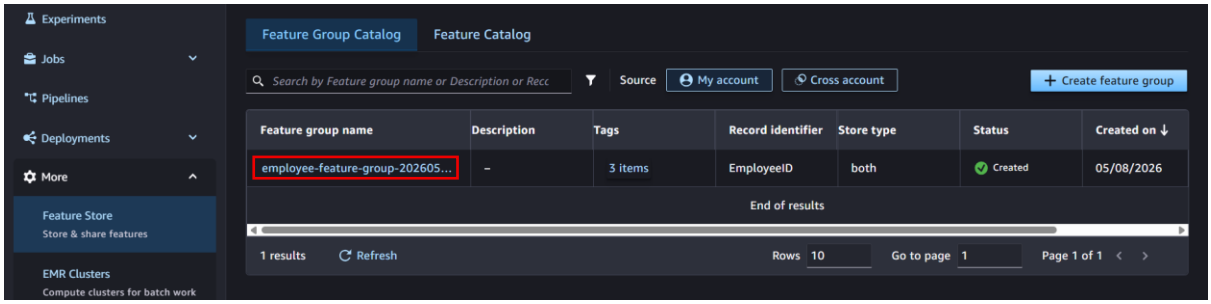
- Upload dataset to S3:
- Create a folder (e.g., input-data)
- Upload the CSV there



- Run the cell One by one.
- It prepares and cleans employee data, converts categorical values into numerical format, and stores features using SageMaker Feature Store.
- Note: If your feature group creation cell gives the errors and ask for S3 Bucket permission. Go to your IAM Role and give permission.
- Don't run the next cell until your previous cell ran successfully. Wait until you feature group is created and data is ingested.

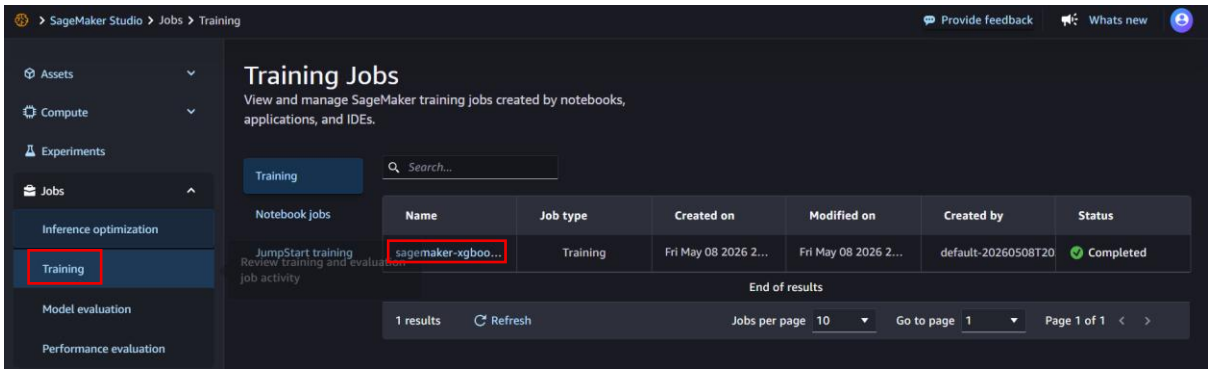
```
[5]: # Check the status of the Feature Group
status = feature_group.describe().get("FeatureGroupStatus")
print(f"Feature Group Status: {status}")

if status == "Created":
    print("Feature Group is Created and ready for use. Proceeding with ingestion...")
    # Ingest data into the Feature Store
    feature_group.ingest(data_frame=employee_df, max_workers=3, wait=True)
    print('Data ingested into Feature Store.')
```

```
Feature Group Status: Created
Feature Group is Created and ready for use. Proceeding with ingestion...
Data ingested into Feature Store.
```

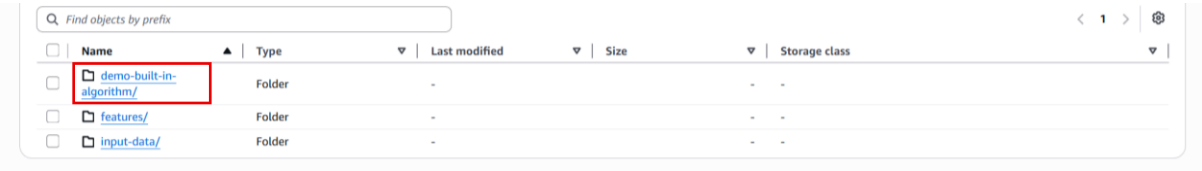


- The dataset is split into training and validation sets using Scikit-learn and then saved as train.csv and validation.csv files for model training.

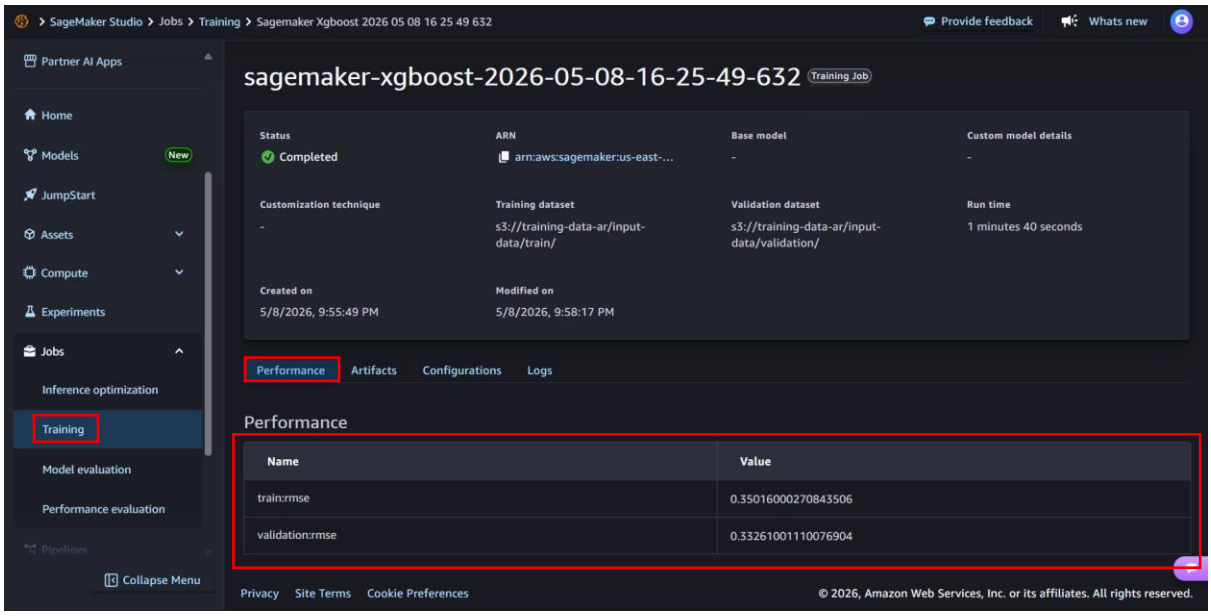
- The processed training and validation datasets are uploaded to Amazon S3, where SageMaker accesses them for XGBoost model training. The notebook configures hyperparameters, sets up the SageMaker training environment, and trains an employee attrition prediction model using AWS services like SageMaker, S3, and Feature Store.



- You can see the output in the 'demo-built-in-algorithm' folder.

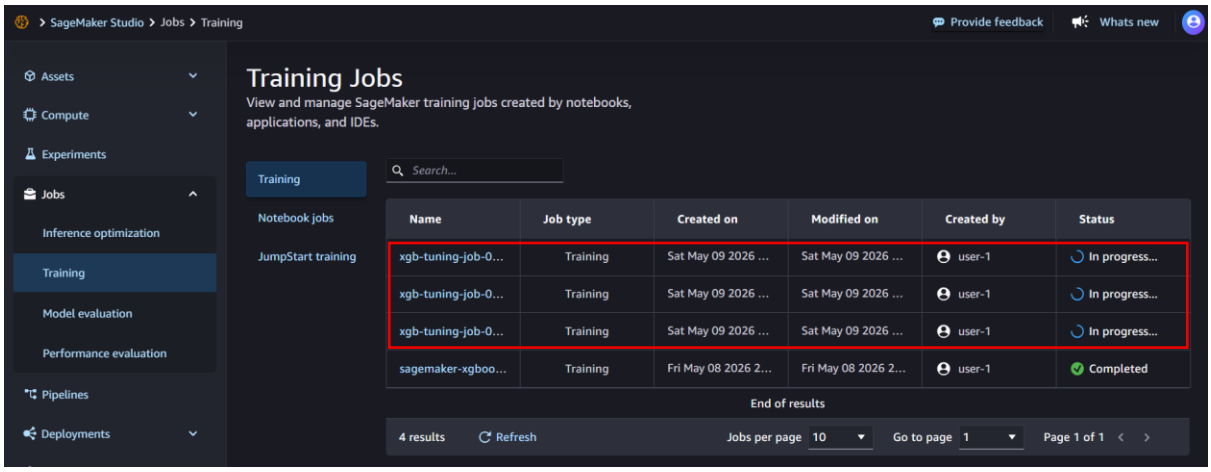


- This is the performance of the model that we have trained



- This step configures hyperparameter tuning in SageMaker by setting up the XGBoost estimator, defining hyperparameter ranges, and choosing RMSE as the optimization metric. SageMaker automatically runs multiple parallel training jobs, evaluates different parameter combinations, and identifies the best-performing hyperparameters for retraining the final model.
- In the below screenshot, It created three training jobs initially because it has configured the tuner to run 3 jobs in parallel.

- In SageMaker hyperparameter tuning, there are two important settings:
- max_jobs → total number of training jobs to try
- max_parallel_jobs → how many can run at the same time
- This means:
- SageMaker will try up to 20 different hyperparameter combinations
- But only 3 training jobs run simultaneously



- Then we will find and inspect the best model produced by a SageMaker Hyperparameter Tuning Job.

```
# Replace with your actual tuning job name
tuning_job_name = 'your-tuning-job-name'   # Change your training job name here
```

- Among all trainings, this is the best job

```
Best Training Job: {'TrainingJobName': 'xgb-tuning-job-036-64c3f84e', 'TrainingJobArn': 'arn:aws:sagemaker:us-east-1:463646775279:training-job/xg
b-tuning-job-036-64c3f84e', 'CreationTime': datetime.datetime(2026, 5, 8, 18, 42, 43, tzinfo=tzlocal()), 'TrainingStartTime': datetime.datetime(2
```

- Then we have to retrieve the best hyperparameters found during hyperparameter tuning.
- The main purpose is:
- After SageMaker tries many different parameter combinations, this code extracts the winning configuration so you can reuse it.

```
Best Hyperparameters: {'_tuning_objective_metric': 'validation:rmse', 'eta': '0.46441314308991577', 'gamma': '1.212371863280548', 'max_depth':
'9', 'min_child_weight': '10', 'num_round': '50', 'objective': 'reg:squarederror', 'subsample': '0.9695241877646337'}
```